

UNIVERSIDADE FEDERAL DE SERGIPE
CENTRO DE CIÊNCIAS EXATAS E TECNOLOGIA
DEPARTAMENTO DE COMPUTAÇÃO

Sistemas Operacionais - 2026.2
AV3 — Virtualização, contêineres, namespaces e cgroups.

Enzo Emanuel Costa Maia
Filipe de Carvalho Godoy
Giulian Fábio Bastos Amorim Lima
José Fagner Silva Junqueira
Katyane dos Santos
Rauany Ingrid Santos de Jesus
Vênisson Cardoso dos Santos

São Cristóvão, Sergipe
2026

Nota: Todos os códigos-fonte, prompts e demais artefatos de análise utilizados para a elaboração deste tutorial estão disponíveis publicamente em nosso repositório no GitHub: https://github.com/fagnerjunqueira/SO_2026_2_AV3_Costa_Enzo_Jesus_Rauany__Godoy_Felipe_Lima_Giulian_Junqueira_Jos-_Santos_Katyane

Nota: Link para a pasta com vídeo do grupo: AT3 - VIDEO.mp4

Ambiente e diagrama (Parte A)

Responsável — Enzo Emanuel Maia Costa

1. Metodologia e Ambiente

A coleta da Parte A identificou o host, o ambiente Linux usado pelo Docker e a configuração do daemon. No host foram consultados a versão do Windows, a arquitetura, a quantidade de processadores lógicos e a memória física. Na distribuição Ubuntu do WSL2 foram executados uname -a, a leitura de /etc/os-release e nproc. No Docker foram consultados docker version e docker info; em seguida, a imagem python:3.12-slim foi baixada e inspecionada para registrar seu digest.

O host observado era Windows 10 Home Single Language, 64 bits, build 19045, com 8 processadores lógicos e 8.438.841.344 bytes de memória física reportada. A distribuição Linux era Ubuntu 24.04.1 LTS no WSL2. O kernel identificado foi Linux 6.18.58.1-microsoft-standard-WSL2, e nproc reportou 8 CPUs.

O daemon estava no contexto desktop-linux e era fornecido pelo Docker Desktop 4.95.0, com Docker Engine 29.8.2 (linux/amd64). O docker info identificou o daemon como docker-desktop, reportou 8 CPUs e 3,754 GiB de memória, e indicou cgroups v2 com driver cgroupfs.

2. Diagama de Camadas

HARDWARE x86_64 — 8 CPUs lógicas; aproximadamente 7,86 GiB de RAM reportados

↓

HOST — Windows 10 Home Single Language (build 19045)

↓

CAMADA VIRTUALIZADA — máquina virtual utilitária gerenciada pelo WSL2

├── Kernel Linux: 6.18.58.1-microsoft-standard-WSL2

├── Distribuição Ubuntu 24.04.1 LTS: shell usado para os comandos Linux

└── Docker Desktop 4.95.0 / Engine 29.8.2: daemon Linux
↓
CONTÊINERES LINUX — isolamento por namespaces e controle de recursos por cgroups

Os valores de CPU e memória do daemon são os recursos reportados ao Docker Engine; não devem ser confundidos com as informações físicas do host Windows.

Questão 1 — Onde está o Kernel compartilhado?

Os contêineres Linux compartilham o kernel Linux disponibilizado pelo backend WSL2, identificado como 6.18.58.1-microsoft-standard-WSL2. Eles não executam um kernel próprio por contêiner e não usam o kernel do Windows. O isolamento de processos e recursos é implementado por mecanismos do kernel, como namespaces e cgroups.
Sim, existe uma camada virtualizada intermediária: a máquina virtual utilitária gerenciada pelo WSL2. Ela não é uma VM tradicional configurada separadamente pelo usuário. A distribuição Ubuntu foi usada como ambiente de shell; o daemon que executa os contêineres pertence ao backend Linux do Docker Desktop.

Evidências de namespaces

Responsável — José Junqueira

1. Metodologia

O experimento comparou os namespaces de processos (PID), montagem (mount) e rede (network) de dois contêineres Docker executados simultaneamente. A coleta foi realizada no Docker Desktop 4.87.0 sobre WSL2, com o kernel Linux 6.18.33.2-microsoft-standard-WSL2 e cgroups v2 com o driver cgroupfs.

Os contêineres so_av3_fagner_cntA (A) e so_av3_fagner_cntB (B) foram criados a partir da mesma imagem, fixada por digest na variável IMAGEM_LAB.

Imagem utilizada
python@sha256:5e2bdb1b0d542d5039cffb8181c65930093949194a65c365dfb1b0fe13a4fa5b

O script atividade3/scripts/evidencia_ns_mount.sh realizou a coleta em uma única execução compartilhada com a Parte D, sob responsabilidade de Giulian Lima. Enquanto os contêineres estavam ativos, foram obtidos o PID no host do daemon, o nome do processo com PID 1 interno e os handles dos três namespaces. Os comandos abaixo foram aplicados a cada contêiner, substituindo NOME pelo nome correspondente.

```
None
docker inspect --format 'PID_host={{.State.Pid}}' NOME
docker exec NOME sh -c 'cat /proc/1/comm'
docker exec NOME readlink /proc/1/ns/pid
docker exec NOME readlink /proc/1/ns/mnt
docker exec NOME readlink /proc/1/ns/net
```

2. Resultados

O processo sleep apareceu como PID 1 dentro dos dois contêineres. Na visão do host do daemon, o Docker registrou o PID 6633 para o contêiner A e o PID 6679 para o contêiner B.

Tabela 1 — Comparação dos namespaces dos contêineres A e B

Recurso	Handle em A	Handle em B	Comparação	Interpretação
Processos (PID)	pid:[4026532869]	pid:[4026532945]	Diferentes	Os processos pertencem a namespaces de PID distintos.
Montagem (mount)	mnt:[4026532401]	mnt:[4026532942]	Diferentes	Os processos pertencem a namespaces de montagem distintos.
Rede (network)	net:[4026532871]	net:[4026532947]	Diferentes	Os processos pertencem a namespaces de rede distintos.

Fonte da Tabela 1 — atividade3/evidencias/evidencia_ns_mount_20261109_203438.log

3. Por que os dois contêineres têm PID 1?

Cada namespace de PID tem sua própria numeração de processos. O primeiro processo de um novo namespace recebe o PID 1 naquele espaço, enquanto o mesmo processo pode ter outro PID quando observado de um namespace ancestral [NS].

Neste experimento, cada instância de sleep apareceu como PID 1 dentro do seu contêiner. Na visão do host do daemon, as duas instâncias receberam números diferentes, 6633 e 6679. A leitura de /proc/1/comm mostrou o nome do processo em cada contêiner, e os handles distintos em /proc/1/ns/pid identificaram os dois namespaces aos quais esses processos pertenciam.

4. Questão 2 — O que os handles demonstram?

Os handles são os identificadores exibidos pelos links em /proc/1/ns/. A Tabela 1

mostra que, para cada tipo comparado, o processo do contêiner A estava associado a uma instância de namespace diferente da instância do contêiner B [NS].

Essa comparação estabelece a separação dos namespaces, enquanto o comportamento de cada recurso exige evidências próprias. O alcance de rede depende da configuração e precisa ser verificado por um teste de conectividade. Dois namespaces de montagem podem incluir o mesmo diretório do host, e a permissão de escrita depende de como ele foi montado e das permissões aplicáveis. A Parte D apresenta a verificação de escrita em /dados, e a Parte C trata dos limites de CPU e memória. Avaliar a proteção contra uma fuga do contêiner exige analisar também a configuração de segurança e o kernel compartilhado.

5. Questão 3 — Namespace, cgroup, identidade e política de acesso

Conceito	Função	Exemplo no laboratório
Namespace	Define a visão que um processo tem de determinados recursos do sistema [NS].	Na Parte B, cada instância de sleep aparece como PID 1 dentro do seu contêiner.
Cgroup	Agrupa processos para contabilizar e controlar o uso de recursos, como CPU e memória [CG, RES].	Na Parte C, cpu.max é consultado para verificar a quota de CPU.
Identidade	Identifica o usuário e os grupos sob os quais o processo executa, por meio de UID e GID.	Na Parte D, o teste é executado com um usuário cujo UID é diferente de zero.
Política de acesso	Estabelece quais operações são permitidas sobre um recurso.	Na Parte D, a opção :ro permite a leitura de /dados e impede a escrita por essa montagem.

Na Parte B, a verificação experimental consistiu em comparar os três namespaces e observar a numeração dos processos nas visões interna e do host do daemon.

Referências

[NS] Linux man-pages. *namespaces(7) — Overview of Linux namespaces*. Documentação

[CG] Linux Kernel Documentation. *Control Group v2*. Documentação

[RES] Docker Documentation. *Resource constraints*. Documentação

Parte C: Experimento de cgroups — Vênisson e Katyane

Autores — Vênisson Cardoso dos Santos e Katyane dos Santos

1. Objetivo e participação dos autores

Esta etapa da atividade investiga como os cgroups v2 restringem o consumo de recursos de uma carga determinística, sem transformar o contêiner em uma máquina virtual independente. Compara-se o mesmo trabalho sob duas quotas de CPU, verificando correção, duração, trabalho concluído por segundo, contadores do kernel e memória observável. A organização segue os requisitos da Parte C e das métricas mínimas do enunciado revisado, especialmente os itens 6 e 6.1 e a questão sobre mediana, variação e ameaça à validade (páginas 3 e 4).

Vênisson Cardoso dos Santos — conduziu a preparação e a execução local no computador documentado, organizando registros brutos, tabelas, gráficos e a publicação das evidências. **Katyane dos Santos —** participou como coautora, replicou o procedimento experimental e acompanhou o trabalho remotamente. A colaboração de Katyane é identificada separadamente da operação local realizada por Vênisson.

As seis medições quantitativas apresentadas pertencem ao lote 20261110T144547Z_2382, obtido no computador de Vênisson. Os registros separados da replicação de Katyane não integram esse lote e, por isso, não são somados às amostras nem utilizados nas medianas. A autoria compartilhada não altera o tamanho da amostra: n=3 por condição.

2. Ambiente, parâmetros e procedimento

O host era Windows 11, com processador AMD Ryzen 7 7435HS. A distribuição de acesso era Ubuntu 24.04.5 LTS no WSL2; o daemon Linux era fornecido pelo Docker Desktop 4.99.0, com Engine 29.9.3, kernel 6.18.54.1-microsoft-standard-WSL2, arquitetura linux/amd64 e cgroups v2 com driver cgroupfs. O Engine informou 16 CPUs lógicas e 4.009.660.416 bytes de memória (aproximadamente 3,734 GiB). O Python utilizado pela carga era o do contêiner, versão 3.12.16.

A imagem foi fixada em python@sha256:5e2bdb1b0d542d5039cffb8181c65930093949194a65c365dfb1b0fe13a4fa5b, o mesmo digest registrado na Parte B de Fagner. Esse alinhamento é parcial: distribuição, versões de kernel e Docker e número de CPUs diferem da máquina de referência. Não se presume identidade com o ambiente ainda não publicado de Enzo.

C1 recebeu quota de 0,5 CPU e C2, de 1 CPU. Em ambas foram configurados 256 MiB de RAM, RAM+swap total de 256 MiB, pids-limit=64 e rede desativada. Cada carga realizou exatamente 20.000.000 de iterações do mesmo algoritmo de

checksum. A quantidade de trabalho e a imagem foram mantidas nas seis repetições, sem alteração de N durante a comparação.

O coletor permaneceu no mesmo contêiner da carga, com leitura nominal de RSS a cada 50 ms. O limite interno era 60 s; o executor utilizava timeout externo de 90 s. As condições foram alternadas na ordem C1/R1, C2/R1, C1/R2, C2/R2, C1/R3 e C2/R3. A cópia do executor foi adaptada para docker sem sudo; o código da carga e do coletor foi preservado.

Os limites foram confirmados nos seis JSON: cpu.max=50000 100000 em C1 e 100000 100000 em C2, memory.max=268435456 e pids.max=64. O primeiro par representa quota agregada de 50 ms por período de 100 ms; não é afinidade nem reserva de núcleo. Um contêiner diagnóstico separado confirmou memory.swap.max=0; esse campo não foi armazenado nos JSON pelo coletor original.

3. Correção e métricas por execução

Tabela C-VK-1 — Trabalho, correção e tempos das seis execuções. Wall é a duração interna da carga; CPU é o tempo de processador do subprocesso. A vazão foi calculada por iterações concluídas ÷ wall da carga, usando os valores completos dos JSON. Trata-se de iterações/s, não de tokens/s. Os valores de vazão são exibidos arredondados para a unidade.

Exec.	Iterações	Checksum	Status	Wall (s)	CPU (s)	Iterações/s
C1_R1	20.000.000	4.864.000	sucesso	3,895119	1,935240	5.134.631
C2_R1	20.000.000	4.864.000	sucesso	2,136695	1,971771	9.360.250
C1_R2	20.000.000	4.864.000	sucesso	3,811518	1,895045	5.247.253
C2_R2	20.000.000	4.864.000	sucesso	2,165725	1,998623	9.234.783
C1_R3	20.000.000	4.864.000	sucesso	3,820625	1,901616	5.234.746
C2_R3	20.000.000	4.864.000	sucesso	2,080971	1,919330	9.610.898

Todas as execuções retornaram status de sucesso, código de saída zero, sem timeout e sem stderr interno ou externo. O checksum 4.864.000 foi o mesmo nas seis amostras e foi conferido também por uma expressão independente da soma dos quadrados módulo 1.000.000.007. Assim, a comparação manteve trabalho e correção verificáveis, em vez de comparar apenas durações de tarefas diferentes.

4. Mediana, variação e trabalho por segundo

Tabela C-VK-2 — Síntese de três repetições por condição. A variação é apresentada pelo intervalo mínimo–máximo e pelo desvio-padrão amostral (DP, denominador n−1). A vazão mediana é a mediana das três razões individuais N/wall, não uma taxa calculada com tempo de monitoramento.

Condição	Wall med. (s)	Wall mín.–máx. (s)	DP (s)	Vazão med. (it/s)
C1	3,820625	3,811518–3,895119	0,045865	5.234.746
C2	2,136695	2,080971–2,165725	0,043072	9.360.250

A mediana do wall da carga passou de 3,820625 s em C1 para 2,136695 s em C2. A razão C1/C2 foi 1,788; portanto, C2 teve wall mediano 44,1% menor neste lote. A vazão mediana passou de 5.234.746 para 9.360.250 iterações/s, aproximadamente a mesma razão de aumento, por se tratar de trabalho fixo.

O coeficiente de variação do wall (DP dividido pela média) foi 1,19% em C1 e 2,02% em C2. A variação entre repetições não desaparece com a alternância. Com n=3, essas medidas são descritivas e não sustentam uma previsão geral de desempenho nem a exigência de um ganho exatamente igual a dois.

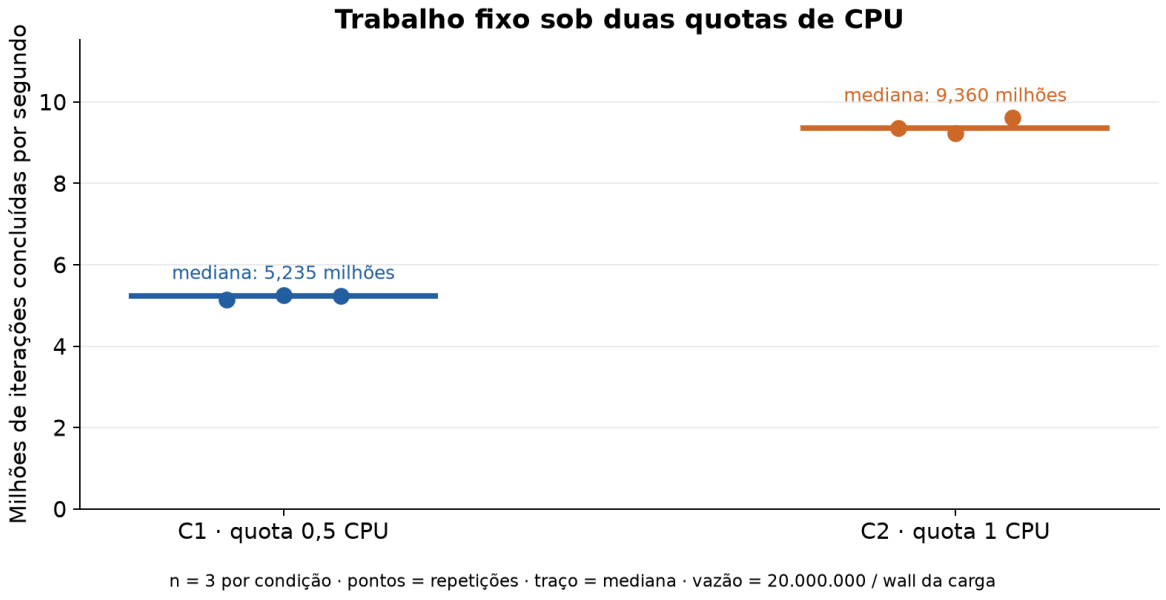


Figura C-VK-1 — Vazão de trabalho fixo por quota de CPU. Eixo vertical em milhões de iterações/s; n=3 por condição. Os pontos representam as repetições e o traço representa a mediana. Fonte: JSON do lote 20261110T144547Z_2382; cálculos dos autores com conferência dos dados brutos.

Uma observação importante é a diferença entre tempo de CPU e tempo de parede. As medianas de CPU do processo foram 1,901616 s em C1 e 1,971771 s em C2, próximas entre si, enquanto os tempos de parede diferiram mais. Em C1, a razão CPU/wall de cada repetição ficou aproximadamente entre 0,497 e 0,498,

comportamento compatível com uma quota de meia CPU nas condições observadas.

O monitoramento não equivale ao wall da aplicação: suas medianas foram 3,986253 s em C1 e 2,323793 s em C2. Ele inclui o acompanhamento do subprocesso e a detecção do encerramento. Por isso, foi preservado como métrica própria e não usado no denominador da vazão de trabalho.

5. Observações dos contadores de CPU

Tabela C-VK-3 — Deltas entre as leituras anterior e posterior da carga. CPU cgroup é usage_usec convertido para segundos e inclui coletor mais carga. Throttled é a quantidade de períodos em que o kernel contabilizou restrição; throttling (s) é throttled_usec convertido para segundos.

Exec.	CPU cgroup (s)	Períodos	Throttled	Throttling (s)	Per. throttled (%)
C1_R1	2,168239	44	44	1,849186	100,0%
C2_R1	2,125212	24	0	0,000000	0,0%
C1_R2	1,990044	40	40	1,690848	100,0%
C2_R2	2,115114	23	0	0,000000	0,0%
C1_R3	2,000654	40	40	1,723812	100,0%
C2_R3	2,083845	23	0	0,000000	0,0%

A observação mais clara é que C1 apresentou "throttling” nos 124 de 124 períodos contabilizados, somadas as três repetições. Em C2, foram zero de 70 períodos. Esse resultado é compatível com a maior restrição de C1, mas o percentual de períodos throttled não é o percentual do tempo total da aplicação. O contador de throttling também não deve ser somado ao wall para obter uma duração total.

As medianas de CPU do cgroup foram 2,000654 s em C1 e 2,115114 s em C2. A quantidade de trabalho foi idêntica, e a quota limita sua taxa de consumo; isso ajuda a interpretar o aumento do wall em C1 sem tratar CPU consumida e duração como a mesma grandeza. Como o coletor compartilha o cgroup, esses valores não representam exclusivamente a carga.

6. Memória e amostragem

Tabela C-VK-4 — Três métricas com escopos distintos: RSS é do processo da carga; memory.current mede o cgroup no instante da leitura final; memory.peak registra o máximo do cgroup desde sua criação. MiB foi calculado por bytes ÷ 1.048.576; o RSS informado em KiB foi dividido por 1.024.

Exec.	RSS processo (MiB)	memory.current final (MiB)	memory.peak (MiB)
C1_R1	15,84	15,61	25,03
C2_R1	15,81	12,61	22,01
C1_R2	15,69	12,61	22,37
C2_R2	15,76	15,72	25,38
C1_R3	15,66	13,17	22,61
C2_R3	15,75	12,89	22,28

O maior memory.peak foi 25,37890625 MiB, aproximadamente 9,91% dos 256 MiB configurados. Os valores confirmam que a carga não se aproximou do teto, mas não constituem teste de pressão de memória ou de OOM. Da mesma forma, pids.max=64 foi lido, sem teste de saturação de tarefas. Não há base para chamar RSS, current e peak de “RAM média”, nem para localizar o instante do pico a partir de uma única leitura de peak.

A razão aproximada monitoramento/amostras de RSS ficou entre 0,052041 e 0,053656 s por amostra. O verificador de Filipe não classificou nenhuma das seis execuções como degradadas segundo sua regra de ultrapassar duas vezes os 50 ms nominais. Entretanto, essa razão não mede os intervalos reais de uma série temporal: sem timestamps individuais, não se demonstra ausência de jitter ou regularidade perfeita da coleta.

7. Principais ameaças à validade e limites da conclusão

A principal ameaça à validade é o ruído do host não controlado: quatro contêineres do projeto clinicaescola permaneceram ativos antes e depois do lote, além dos demais aplicativos do usuário. Isso pode afetar a disponibilidade de CPU, escalonamento e duração. A alternância C1/C2 reduz o efeito de uma tendência temporal simples, mas não isola o host nem substitui randomização e mais repetições.

Outra limitação é o coletor dentro do cgroup da carga. Ele consome parte da quota e pode sofrer restrição junto com o processo observado. Essa interferência justifica distinguir métricas do grupo e da aplicação; não permite atribuir a ela, isoladamente,

a causa de toda diferença de tempo. A leitura de cpu.max confirma o limite local, mas este lote não inclui inspeção completa de todos os limites ancestrais.

O enunciado também pede um piloto pequeno antes de congelar N. Nos artefatos consultados deste lote não foi localizado um registro específico desse piloto. Portanto, documenta-se o N efetivamente usado e preservado nas seis execuções, sem afirmar que um piloto não registrado foi realizado. Tentativas e versões anteriores continuam rastreáveis no histórico e nos arquivos locais; não foram tratadas como novas repetições para melhorar as estatísticas.

O resultado sustenta que a restrição de CPU funcionou nas condições medidas: limites lidos do kernel, trabalho correto, maior wall em C1 e ocorrência de throttling em C1. Ele não prova isolamento absoluto, desempenho idêntico em máquinas diferentes, aplicação de limites de VRAM ou controle de todos os recursos do host. No Docker Desktop/WSL2 há uma camada Linux virtualizada intermediária; os contêineres usam o kernel desse ambiente, em vez de terem kernels próprios.

8. Integração com Filipe, reprodução e referências

O trabalho de Filipe de Carvalho Godoy complementa esta contribuição pela confirmação dos limites e distinção entre métricas. Seu script scripts/verifica_limites.py foi aplicado ao novo lote sem modificações. Os números desta seção são resultados dessa aplicação aos JSON atuais, e não uma nova redação ou aprovação atribuída a Filipe. A seção seguinte conserva integralmente o texto e as tabelas que ele produziu para o lote que analisou; não se agregam os dois conjuntos como uma única amostra.

As evidências publicadas incluem os seis JSON, stderr, manifesto, CSV consolidado, gráficos, consultas de ambiente e SHA256SUMS. Referência fixa de código e autoria: commit 2e9e93cb48a108cfa38351e6b28b5808e7a87390 da branch feature/parte-c-venisson. A origem da imagem e seus parâmetros são registrados no manifesto do lote.

Reprodução a partir da raiz do repositório: bash scripts/parte_c_venisson/run_experimentos.sh. O executor cria um diretório próprio para cada lote. Para conferir o lote publicado: python3 scripts/verifica_limites.py evidencias/parte_c_venisson/resultados/20261110T144547Z_2382 --csv verificacao_novo_lote.csv. O segundo comando gera outra tabela e não deve ser usado para sobrescrever a tabela original de Filipe.

Houve assistência de IA no apoio à adaptação do executor, à organização dos dados, aos cálculos e à redação. As saídas foram conferidas com os JSON reais, o checksum, os limites lidos e os hashes; texto gerado não foi utilizado como medição. Vênisson e Katyane devem revisar e explicar o procedimento e suas contribuições na apresentação.

Referências desta seção: enunciado revisado da Atividade 3, itens 6, 6.1, 8, 10 e 12; JSON e CSV do lote 20261110T144547Z_2382; script de verificação e relatório

original de Filipe; documentação Control Group v2 do Linux Kernel e Resource constraints do Docker.

NOTA DE INTEGRAÇÃO — LOTE HISTÓRICO: as tabelas e hipóteses da seção de Filipe abaixo pertencem ao lote anterior, preservado no commit 8f47ee56cc49394418865cb2ffb286e807c6415c. Não combinar estes valores com o lote WSL2 20261110T144547Z_2382 da seção de Vênisson e Katyane. No lote vigente, os limites conferem nas seis execuções, CPU do cgroup mediana 2,000654/2,115114 s e throttling C1=124/124 períodos, C2=0/70. A razão monitoramento/amostras é apenas densidade aproximada, sem timestamps individuais.

Parte C: Confirmação dos limites e métricas de cgroup

Responsável — Filipe de Carvalho Godoy

1. Objetivo

Verificar se os limites solicitados na linha de comando do Docker foram efetivamente aplicados pelo kernel, consolidar os contadores de CPU e as métricas de memória do cgroup nas seis execuções da Parte C, e responder à Questão 4.

2. Metodologia

A execução das seis repetições foi realizada por Vênisson Cardoso dos Santos. Esta seção trabalha sobre os mesmos registros brutos, sem reexecução, por meio do script `scripts/verifica_limites.py`.

A flag `--cpus` é uma solicitação ao runtime: descreve a intenção, não o estado do kernel. Entre a flag e o enforcement estão o runtime, o driver de cgroup e a disponibilidade dos controladores no cgroup ancestral, e qualquer um pode falhar silenciosamente. A prova de que o limite existe é a leitura de `/sys/fs/cgroup/cpu.max`, `memory.max` e `pids.max` de dentro do contêiner, realizada antes de cada execução.

Imagem utilizada:
`python@sha256:4840d514c583a4d3d8cad3940e68dee6f0f6f3635c12fcd0a3d74c21a5b93a54`

3. Confirmação dos limites

Tabela 1 — Limites lidos do cgroup nas seis execuções

Execução	cpu.max	Quota derivada	memory.max	pids.max	Confere
C1_R1	50000 100000	0,50 CPU	256 MiB	64	Sim
C1_R2	50000 100000	0,50 CPU	256 MiB	64	Sim
C1_R3	50000 100000	0,50 CPU	256 MiB	64	Sim
C2_R1	100000 100000	1,00 CPU	256 MiB	64	Sim
C2_R2	100000 100000	1,00 CPU	256 MiB	64	Sim
C2_R3	100000 100000	1,00 CPU	256 MiB	64	Sim

Fonte: `evidencias/limites_cgroup.csv`

O valor de `cpu.max` é um par: quota e período, em microssegundos. O par `50000 100000` significa 50 ms de tempo de CPU agregado a cada período de 100 ms. A quota apresentada foi derivada desse par, e não copiada da flag. Trata-se de tempo agregado, não de afinidade ou reserva de núcleo: o cgroup pode executar em qualquer CPU disponível, e o kernel contabiliza a soma do tempo consumido por todas as suas tarefas dentro de cada janela.

O valor 268435456 bytes de `memory.max` corresponde a 256 MiB. Com `--memory-swap=256m` igual à memória, o contêiner fica sem swap adicional.

4. Contadores de CPU

Tabela 2 — Deltas de `cpu.stat` entre as leituras anterior e posterior à carga

Execução	`usage_usec` Δ	Janela	Períodos	Throttled	%	`throttled_usec` Δ
C1_R1	3,571 s	10,60 s	106	105	99%	13,855 s
C1_R2	4,058 s	11,90 s	119	119	100 %	7,877 s
C1_R3	3,700 s	10,50 s	105	104	99%	6,888 s
C2_R1	4,940 s	5,20 s	52	27	52%	0,383 s
C2_R2	3,940 s	7,80 s	78	75	96%	7,437 s
C2_R3	3,755 s	7,30 s	73	70	96%	3,509 s

A janela corresponde a `nr_periods × 100 ms`, o tempo de parede coberto pelos períodos contabilizados.

Houve throttling em todas as execuções, com intensidade maior em C1: entre 99% e 100% dos períodos, contra 52% a 96% em C2. A direção acompanha a restrição imposta.

O valor de `usage_usec` é praticamente constante entre as condições — mediana de 3,700 s em C1 e 3,940 s em C2. Isso é coerente: o trabalho é o mesmo nas seis execuções, com checksum idêntico de 4.864.000. A quota limita a taxa de consumo, não a quantidade de trabalho a realizar.

O contador `throttled_usec` chega a 13,86 s numa janela de 10,60 s porque soma a espera de todas as tarefas do cgroup; com duas tarefas bloqueadas simultaneamente, o acumulado cresce mais rápido que o relógio.

5. Memória

As três métricas são grandezas distintas: o RSS mede a memória residente do processo da carga; `memory.current` mede o uso do cgroup inteiro no instante da leitura; `memory.peak` registra o maior valor atingido desde a criação do cgroup, incluindo a inicialização do contêiner.

Tabela 3 — Métricas de memória por execução

Execução	RSS do processo	`memory.current` (fim)	`memory.peak`	Teto	% do teto
C1_R1	15,88 MiB	12,00 MiB	21,75 MiB	256 MiB	8,5%
C1_R2	15,75 MiB	11,99 MiB	21,74 MiB	256 MiB	8,5%
C1_R3	15,75 MiB	12,01 MiB	21,75 MiB	256 MiB	8,5%
C2_R1	15,62 MiB	11,77 MiB	21,62 MiB	256 MiB	8,4%
C2_R2	15,85 MiB	12,00 MiB	21,75 MiB	256 MiB	8,5%
C2_R3	15,67 MiB	11,75 MiB	21,48 MiB	256 MiB	8,4%

A ordenação entre as métricas é informativa. O `memory.peak` supera o RSS do processo porque o pico ocorre durante a inicialização, quando o interpretador do coletor e o da carga coexistem. O `memory.current` ao final é menor que ambos porque a carga já havia terminado e suas páginas foram liberadas.

Nenhuma execução passou de 8,5% do teto. O limite está comprovadamente aplicado, mas o experimento não produz evidência sobre o comportamento do

cgroup sob pressão de memória. O mesmo vale para `pids.max`: confirmado em 64, nunca saturado.

6. Questão 4 — Como a quota foi confirmada? Houve throttling?

A quota foi confirmada por leitura de `/sys/fs/cgroup/cpu.max` de dentro de cada contêiner, antes de cada uma das seis execuções, e não pela flag passada ao Docker. Os valores lidos foram `50000 100000` em C1 e `100000 100000` em C2, dos quais se deriva a quota real de 0,5 e 1,0 CPU por período de 100 ms. Os controladores `memory.max` (256 MiB) e `pids.max` (64) foram confirmados pelo mesmo procedimento, com os três conferindo nos seis registros.

Houve throttling, contabilizado pelo kernel em todas as execuções: entre 99% e 100% dos períodos em C1 e entre 52% e 96% em C2.

Relacionando contadores, tempo e trabalho: o trabalho foi idêntico nas seis execuções e o custo em CPU medido pelo kernel foi praticamente constante, o que é coerente, já que a quota limita a taxa de consumo e não a quantidade de trabalho. O que deveria diferir entre as condições é o tempo de parede, e é nesse ponto que as medições se mostram inconsistentes.

Estes dados não permitem afirmar que a quota de 0,5 CPU produziu a desaceleração proporcional esperada na carga. A confirmação dos limites e a ocorrência de throttling estão demonstradas; a magnitude do efeito sobre o tempo de execução, não.

7. Limitação identificada

O coletor de métricas executa dentro do mesmo cgroup da carga que mede, mantendo um laço que lê `/proc/<pid>/status` a cada 50 ms. Isso o torna uma segunda tarefa competindo pela mesma quota e faz com que seja throttled junto com a carga.

Tabela 4 — Intervalo efetivo entre amostras de RSS

Execução	% períodos throttled	Amostras	Intervalo efetivo	Atraso
C1_R1	99%	6	1,768 s	35×
C1_R2	100%	7	1,702 s	34×

C1_R3	99%	13	0,789 s	16×
C2_R1	52%	87	0,054 s	1,1×
C2_R2	96%	6	1,296 s	26×
C2_R3	96%	3	2,346 s	47×

A execução C2_R1 é a única com throttling moderado e a única em que o coletor operou como projetado, com intervalo de 0,054 s contra os 0,05 s programados. Nas demais, com throttling entre 96% e 100%, o intervalo cresceu de 16 a 47 vezes.

Essa contaminação não compromete a confirmação dos limites, que é leitura direta de estado do kernel, nem a validação funcional das execuções. Compromete o uso do tempo de monitoramento como medida de desempenho da carga, e torna os contadores de `cpu.stat` atribuíveis ao conjunto coletor mais carga, não à carga isoladamente.

Permanece uma discordância que estes dados não explicam: a carga reporta tempo de parede aproximadamente igual ao tempo de CPU em todas as execuções, o que seria incompatível com throttling sustentado. Há hipóteses plausíveis — priorização da carga pelo escalonador, ou os eventos de *soft lockup* registrados na máquina virtual —, mas nenhuma pode ser confirmada com seis amostras. Registra-se a divergência entre a medição interna e os contadores do kernel, sem atribuição de causa, sendo estes últimos a fonte preferível por não dependerem do relógio de espaço de usuário.

Para trabalhos futuros, a instrumentação deveria observar o cgroup de fora dele: executando o coletor no host, substituindo a amostragem ativa de RSS por leitura única de `memory.peak`, e medindo o tempo de parede no host. Nenhum desses ajustes foi aplicado ao lote analisado, pois os registros originais foram preservados conforme determina o enunciado.

NOTA DE INTEGRAÇÃO — ANÁLISE HISTÓRICA: a seção de Rauany preservada abaixo usa o lote anterior, com medianas 3,514819/3,674589 s e amplitude 0,637226/0,915566 s. Esses números não pertencem ao lote WSL2 20261110T144547Z_2382. Para a comparação vigente, usar a seção de Vênisson e Katyane: wall mediano 3,820625/2,136695 s; amplitude

0,083601/0,084754 s; vazão mediana 5.234.746/9.360.250 iterações/s, n=3 por condição. A razão das vazões é 1,788. A análise histórica permanece para rastreabilidade.

Parte C: Análise e comparação C1 vs C2

Responsável: Rauany Ingrid Santos de Jesus

1. Objetivo

Comparar as condições C1 (0,5 CPU) e C2 (1,0 CPU) quanto ao trabalho concluído por segundo, e responder à Questão 5: o que aconteceu com a mediana e a variação entre C1 e C2, e qual é a principal ameaça à validade.

2. Metodologia

A análise usa as seis execuções do lote `20261110T144547Z_2382`, conduzidas por Vênisson Cardoso dos Santos e Katyane dos Santos, com os limites confirmados por Filipe de Carvalho Godoy. O ambiente (Windows 11, Ubuntu 24.04.5 no WSL2, Docker Desktop 4.99.0, kernel 6.18.54.1, cgroups v2) e a imagem fixada por digest estão descritos na seção de Vênisson e Katyane. Esta seção trabalha sobre os mesmos registros, sem reexecução.

Em cada execução, a carga realizou 20.000.000 de iterações, com checksum 4.864.000 e status de sucesso nas seis. As condições foram alternadas (C1/R1, C2/R1, C1/R2, C2/R2, C1/R3, C2/R3). O trabalho concluído por segundo é a quantidade de iterações dividida pelo tempo de parede (*wall*) da carga. Essa grandeza mede a taxa de execução e não equivale a tokens por segundo. A variação é expressa pela amplitude (máximo menos mínimo) e pelo desvio-padrão amostral do wall.

3. Resultados

Tabela 1: Registros das seis execuções

Execução	Wall (s)	CPU do processo	CPU/wall	Iterações
C1_R1	3,895119	1,935240	0,497	5.134.631

C2_R1	2,136695	1,971771	0,923	9.360.250
C1_R2	3,811518	1,895045	0,497	5.247.253
C2_R2	2,165725	1,998623	0,923	9.234.783
C1_R3	3,820625	1,901616	0,498	5.234.746
C2_R3	2,080971	1,919330	0,922	9.610.898

Fonte: JSON do lote `20261110T144547Z_2382`. A razão CPU/wall foi calculada a partir das duas colunas anteriores.

Tabela 2: Mediana e variação por condição (n = 3 repetições)

Condição	Wall mediano	Mínimo	Máximo	Amplitude	DP	Vazão mediana
C1 (0,5 CPU)	3,820625	3,811518	3,895119	0,083601	0,045865	5.234.746
C2 (1,0 CPU)	2,136695	2,080971	2,165725	0,084754	0,043072	9.360.250

Gráfico 1: Vazão de trabalho fixo por condição (n = 3; barra = mediana).

4. Questão 5: O que aconteceu com a mediana e a variação entre C1 e C2?

A vazão mediana passou de 5.234.746 iterações/s em C1 para 9.360.250 em C2, uma razão de 1,788. Como o trabalho é fixo, a razão do wall é a mesma, e C2 teve wall mediano 44,1% menor. A quota de C2 é o dobro da de C1, mas o enunciado não exige ganho exato de 2, e o valor observado ficou abaixo disso.

A razão CPU/wall dá uma leitura coerente com a quota. Em C1 ficou em 0,497 a 0,498, o valor esperado para meia CPU. Em C2 ficou em 0,922 a 0,923, perto de uma CPU inteira. O tempo de CPU do processo ficou próximo nas duas condições (medianas de 1,901616 s em C1 e 1,971771 s em C2), como se espera de trabalho fixo: a quota limita a taxa de consumo, não a quantidade de trabalho.

Os contadores de throttling acompanham a direção da restrição: em C1 houve throttling em 124 de 124 períodos, e em C2, em 0 de 70. Essa leitura é compatível com a diferença de tempo, mas o percentual de períodos com throttling não é o percentual do tempo da aplicação, e uma única amostra por condição não permite concluir a causa do resultado.

Quanto à variação, as amplitudes foram praticamente iguais (0,084 s em C1 e 0,085 s em C2). O coeficiente de variação foi de 1,19% em C1 e 2,02% em C2. Em valor absoluto a dispersão é parecida, e em termos relativos é maior em C2 porque o wall é menor. Com n = 3, esses números descrevem as amostras e não estimam a variabilidade do sistema.

5. Principal ameaça à validade

A principal ameaça é o **ruído do host não controlado**. Outros contêineres e aplicativos do computador permaneceram ativos durante o lote, o que pode afetar a CPU disponível e o tempo medido. A alternância entre C1 e C2 reduz o efeito de uma tendência temporal simples, mas não isola o host nem substitui randomização e mais repetições.

Outras ameaças reduzem o alcance da conclusão:

- **Coletor no mesmo cgroup da carga:** ele consome parte da quota e pode ser limitado junto com ela, então os contadores de grupo refletem coletor e carga juntos. Por isso o wall da aplicação, e não o tempo de monitoramento, foi usado na vazão.
- **Tamanho da amostra:** três repetições por condição não sustentam inferência estatística nem previsão geral de desempenho.
- **Cgroups ancestrais:** limites acima do contêiner não foram avaliados.

6. Relação com a infraestrutura de inferência

Em um servidor que compartilha o hardware entre cargas de inferência, a quota de CPU controla a taxa de consumo de cada serviço, sem reservar núcleo nem reduzir o trabalho necessário. O resultado mostra que dobrar a quota reduziu o tempo da carga em 44%, mas não pela metade, e que o efeito deve ser confirmado com os contadores do kernel e não só com a flag passada ao Docker.

7. Limitações

Os resultados vêm de uma carga sintética de CPU, em um único lote de seis execuções, sem modelo de linguagem, GPU ou VRAM. Não se generalizam para inferência real nem para outros ambientes.

Referências

[CG] Linux Kernel Documentation. Control Group v2.
https://docs.kernel.org/admin-guide/cgroup-v2.html
[RES] Docker. Resource constraints.
https://docs.docker.com/engine/containers/resource_constraints/

Parte D: Mount Read-only e Menor Privilégio

Responsável: Giulian Fabio Bastos Amorim Lima

Ambiente individual: Ubuntu 24.04.4 LTS em VirtualBox

1. Objetivo

Executar um contêiner Docker com privilégios reduzidos, utilizando usuário não-root, removendo capabilities e disponibilizando um diretório de documentos somente para leitura.

2. Preparação do ambiente

Instalar e iniciar o Docker:

```
None
sudo apt update
sudo apt install -y docker.io
sudo systemctl enable --now docker
```

Permitir o uso do Docker pelo usuário:

```
None
sudo usermod -aG docker vboxuser
```

Após sair e entrar novamente na sessão, verificar:

```
None
docker info --format 'Cgroup={{.CgroupVersion}}
Driver={{.CgroupDriver}}'
```

Resultado: Cgroup v2 e driver systemd.

3. Preparação dos dados

Baixar a imagem:

```
None
docker pull python:3.12-slim
```

Criar o diretório e o arquivo de teste:

```
None
mkdir -p ~/atividade3/dados_demo

printf '%s\n' 'Documento sintetico da Atividade 3 de
Sistemas Operacionais.' > ~/atividade3/dados_demo/texto.txt
```

Digest da imagem utilizada:

```
None
sha256:4840d514c583a4d3d8cad3940e68dee6f0f6f3635c12fcd0a3d7
4c21a5b93a54
```

4. Execução do contêiner

```
None
IMAGEM_LAB="python:3.12-slim@sha256:4840d514c583a4d3d8cad39
40e68dee6f0f6f3635c12fcd0a3d74c21a5b93a54"

docker run \
  --name giulian-parte-d \
  --network none \
  --user "$(id -u):$(id -g)" \
  --cap-drop ALL \
```

```
  --security-opt no-new-privileges \
  --mount
"type=bind,source=$HOME/atividade3/dados_demo,target=/dados
,readonly" \
  "$IMAGEM_LAB" \
  sh -c '
    id
    cat /dados/texto.txt
    touch /dados/arquivo_teste.txt
    grep -E "^(CapEff|NoNewPrivs):" /proc/self/status
  '
```

5. Verificação das configurações

Inspecionar os privilégios:

```
None
docker inspect --format 'Usuario={{.Config.User}}
CapDrop={{json .HostConfig.CapDrop}} SecurityOpt={{json
.HostConfig.SecurityOpt}}' giulian-parte-d
```

Verificar se o volume é somente leitura:

```
None
docker inspect --format '{{range .Mounts}}{{if eq
.Destination "/dados"}}Destino={{.Destination}}
RW={{.RW}}{{end}}{{end}}' giulian-parte-d
```

6. Resultados obtidos

Teste	Resultado
Usuário	UID/GID 1000, não-root
Leitura do arquivo	Permitida

Escrita no diretório	Negada
Capabilities efetivas	Zero
NoNewPrivs	Habilitado
Volume	`RW=false`

7. Conclusão

Limitar a memória RAM e a quantidade de tarefas evita que processos consumam recursos excessivos, causando sobrecarga ou indisponibilidade do servidor de inferência. Montar o corpus como somente leitura protege os documentos contra alterações e exclusões indevidas. Mesmo com essas medidas, ainda existem riscos como vulnerabilidades no kernel, falhas na aplicação e possíveis escapes do contêiner.